Informe
Grupo 3: Castellano Matias y Heinen Franz.

A la hora de diseñar nuestra herramienta que sea capaz de encontrar qué funciones del conjunto dado son estructuralmente similares a una función objetivo, decidimos implementar la siguiente estructura de diseño:
Organizamos el proyecto estructurando el código fuente dentro del directorio src /.Dentro de él, dividimos en módulos en donde cada uno es encargado de realizar una tarea específica para lograr el objetivo común.

- **Módulo Extractor** (extractor.py): Es el encargado de recorrer de forma recursiva los directorios especificados y parsear los archivos Python utilizando el módulo ast de la librería proporcionada. Además,extrae cada función (FunctionDef o AsyncFunctionDef) junto con sus metadatos (nombre, ruta del archivo, línea de origen y nodo AST del árbol sintáctico).
- **Módulo Normalizador** (normalizer.py): Normaliza el AST de cada función abstrayendo variables y parámetros específicos por nombres genéricos secuenciales (VAR_1, VAR_2,VAR_N.). Para esto, utiliza un diccionario de variables y un contador: al recorrer los nodos, si la variable no está registrada, la agrega vinculada al valor actual del contador. Además, convierte los valores(números, cadenas) al token LIT.
- **Módulo Serializador** (serializer.py): Convierte el árbol AST normalizado en una secuencia lineal de tokens sintácticos y calcula la distancia de Levenshtein(*métrica elegida, explicada debajo*) entre dos secuencias para determinar su porcentaje de similitud y definir si se trata de duplicados o no según un umbral que definimos.
- **Módulo CLI / Interfaz** (cli.py): Este recibe por consola el directorio (--dir), la función objetivo (--funcion como archivo.py:nombre o un bloque de código ingresado) y un valor K que indica la cantidad de resultados deseados (--top). Con estos datos,delega la extracción, normalización y cálculo de similitud a los demás módulos y por último, se encarga de imprimir en pantalla el Top-K ordenado de funciones similares, indicando la ubicación del archivo encontrado, su score de similitud y su clasificación según el umbral (CANDIDATO A DUPLICADO o DISTINTA).

Métrica de Similitud y Definición del Umbral

Elegimos la métrica de Distancia de Levenshtein debido a que al realizar un trabajo práctico en otra materia en la cual implementamos un motor de búsqueda(elasticsearch) notamos que utilizaba esta métrica para identificar palabras similares, con lo cual, nos pareció interesante conocerla más en profundidad y trabajar con ella en un problema similar como el planteado(detectar funciones similares). Notamos que está es muy eficiente debido a que mide la diferencia entre dos secuencias de símbolos mediante el número mínimo de operaciones de edición (inserción, eliminación y sustitución) necesarias para transformar una secuencia en otra.
Fórmula de Similitud Normalizada Implementada:

$$Similitud(S_1, S_2) = 1 - \frac{Distancia_Levenshtein(S_1,S_2)}{\max(longitud(S_1),longitud(S_2))}$$

Aquí S1 y S2 representan las secuencias ordenadas de tokens sintácticos de las funciones comparadas.
Definimos el umbral para que dos funciones sean consideradas duplicadas en: similitud > 75% debido a que al realizar pruebas localmente, detectamos que este valor nos permite identificar los clones de tipo 1 (clones exactos) y tipo 2 (clones renombrados) cuyo cálculo de similitud ronda el 90% - 100%. Además, nos permite detectar clones de tipo 3 livianos (donde se reordenaron líneas independientes o se añadieron pequeñas modificaciones), ya que estos arrojan similitud cercanas al 80%,esto se debe a que el volumen de tokens alterados representa una proporción baja respecto al tamaño total de la función.

Limitaciones Conocidas

Como limitación conocida, la herramienta posee la debilidad de no detectar clones de tipo 3 cuando las funciones evaluadas son de longitud corta, debido a que el valor en el denominador es más pequeño y no permite valores grandes en el numerador como margen. Otra limitación es que no logra detectar clones semánticos (tipo 4). Esto ocurre cuando dos funciones realizan exactamente la misma tarea y producen las mismas salidas, pero están implementadas con estructuras sintácticas diferentes.

Posible Mejora

La herramienta mejoraría mucho si le implementamos un:

- Análisis Dinámico por Pruebas de Entrada-Salida (*Black-Box Execution*): Esto consistirá en ejecutar ambas funciones en un entorno aislado (*sandbox*) con el mismo conjunto de entradas de prueba y comparar sus resultados. Si para todas las entradas se devuelven los mismos valores, el sistema los detecta como clones semánticos, independientemente de su estructura interna. Por ejemplo, podría detectar dos funciones que calculan la suma acumulada de 1 a N, donde una utiliza un bucle for y la otra un bucle while.

Suite de Pruebas

Construimos una suite de pruebas estructurada en la carpeta /tests utilizando pytest:

- Pruebas Positivas (test_positivos.py): Evaluamos que el sistema detecte de manera exitosa: Clones de Tipo 1 (funciones idénticas), Tipo 2 (funciones con variables y parámetros renombrados) y Tipo 3 livianos (reordenamiento de líneas de una función).
- Pruebas Negativas (test_negativos.py): Nos aseguramos la correcta distinción entre funciones con tareas totalmente diferentes en donde la estructura y longitud de cada una sea distinta, además también verificamos la incapacidad de la herramienta frente a clones de Tipo 4.